



Transaction Management



Muhammad Hamiz Mohd Radzi

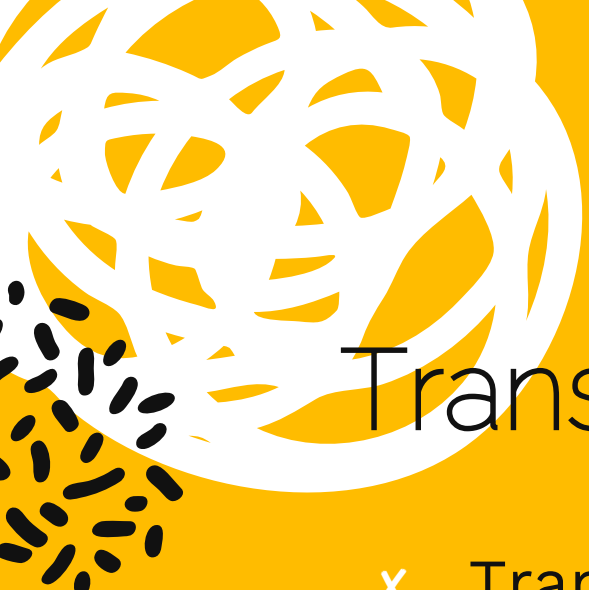
Objectives

- x At the end of this lesson, you should be able to:
- x Explain transaction support, properties of transaction and the database architecture.
- x Explain concurrency control, lost update, uncommitted dependency and inconsistent analysis problems.
- x Explain the serializability schedule, serial and non-serial schedule, non-conflict serializability precedence graph, and its recoverability.
- x Describe the locking, shared and exclusive lock, 2PL's shrinking and growing phase, concurrency problems, solution by using 2PL, cascading rollback, deadlock prevention and detection, and time-stamping concept.



Objectives

- x Describe the granularity data items and its hierarchy.
 - x Explain the database recovery and why it is needed.
 - x Explain the UNDO and REDO recovery mechanism
 - x Describe the log file and checkpoint backup mechanism
 - x Explain the deferred and immediate update, and shadow paging recovery techniques.
- 
- 



Transaction Support

- x Transaction: Action, or series of actions, carried out by user or application, which reads or updates contents of database.
 - x Each operation on database can be considered as transactions.
 - x During transactions, database is transformed from consistent state to another state, although the consistency might be violated.
- 

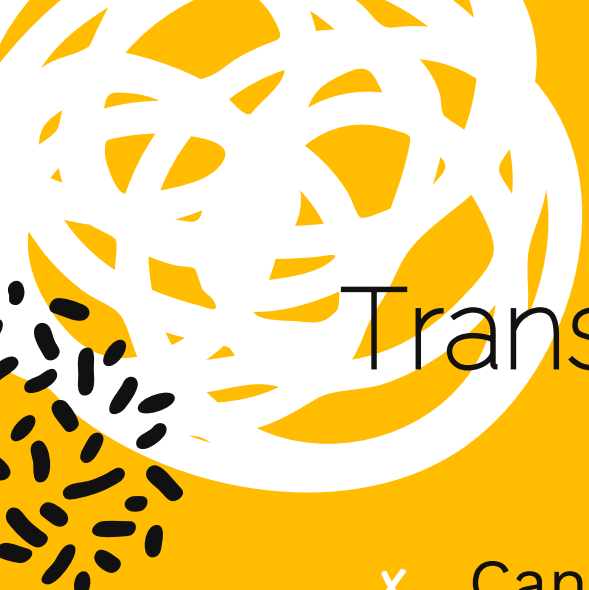
Example of Transaction

```
read(staffNo = x, salary)
salary = salary * 1.1
write(staffNo = x, salary)
```

(a)

```
delete(staffNo = x)
for all PropertyForRent records, pno
begin
    read(propertyNo = pno, staffNo)
    if (staffNo = x) then
        begin
            staffNo = newStaffNo
            write(propertyNo = pno, staffNo)
        end
    end
end
```

(b)

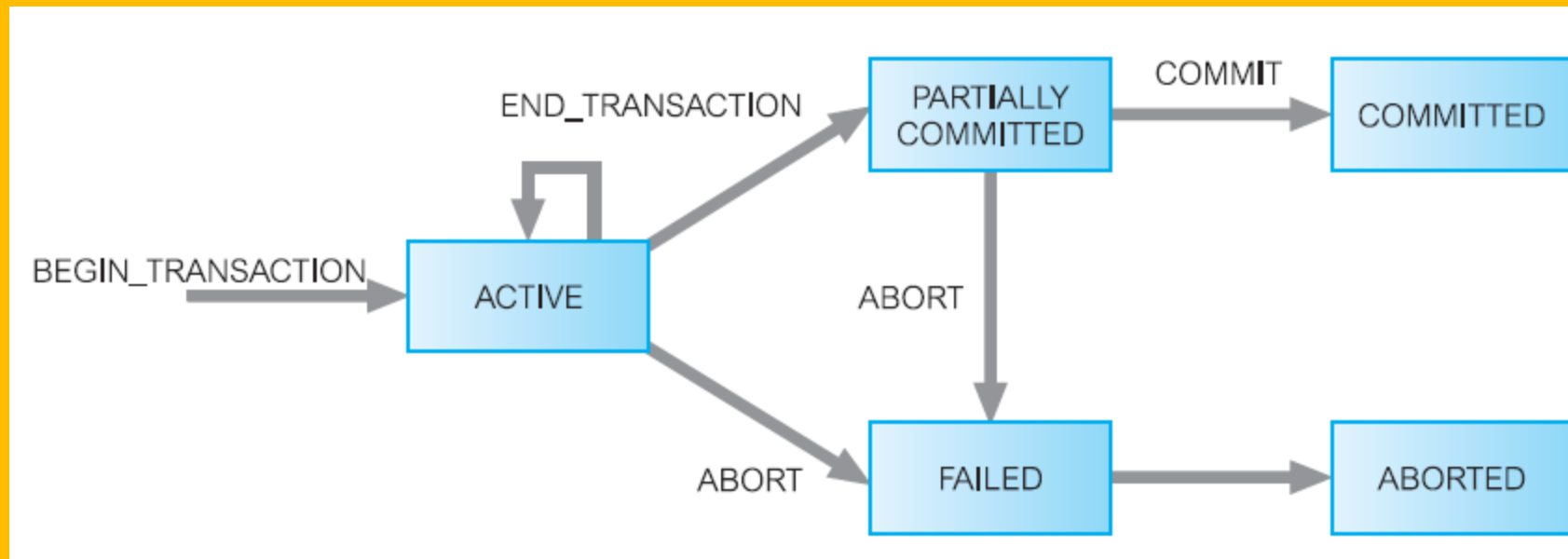


Transaction Support



- x Can have one of two outcomes:
 - x Success - transaction commits and database reaches a new consistent state.
 - x Failure - transaction aborts, and database must be restored to consistent state before it started.
 - x Such a transaction is rolled back or undone.
 - x Committed transaction cannot be aborted.
 - x Aborted transaction that is rolled back can be restarted later.
- 

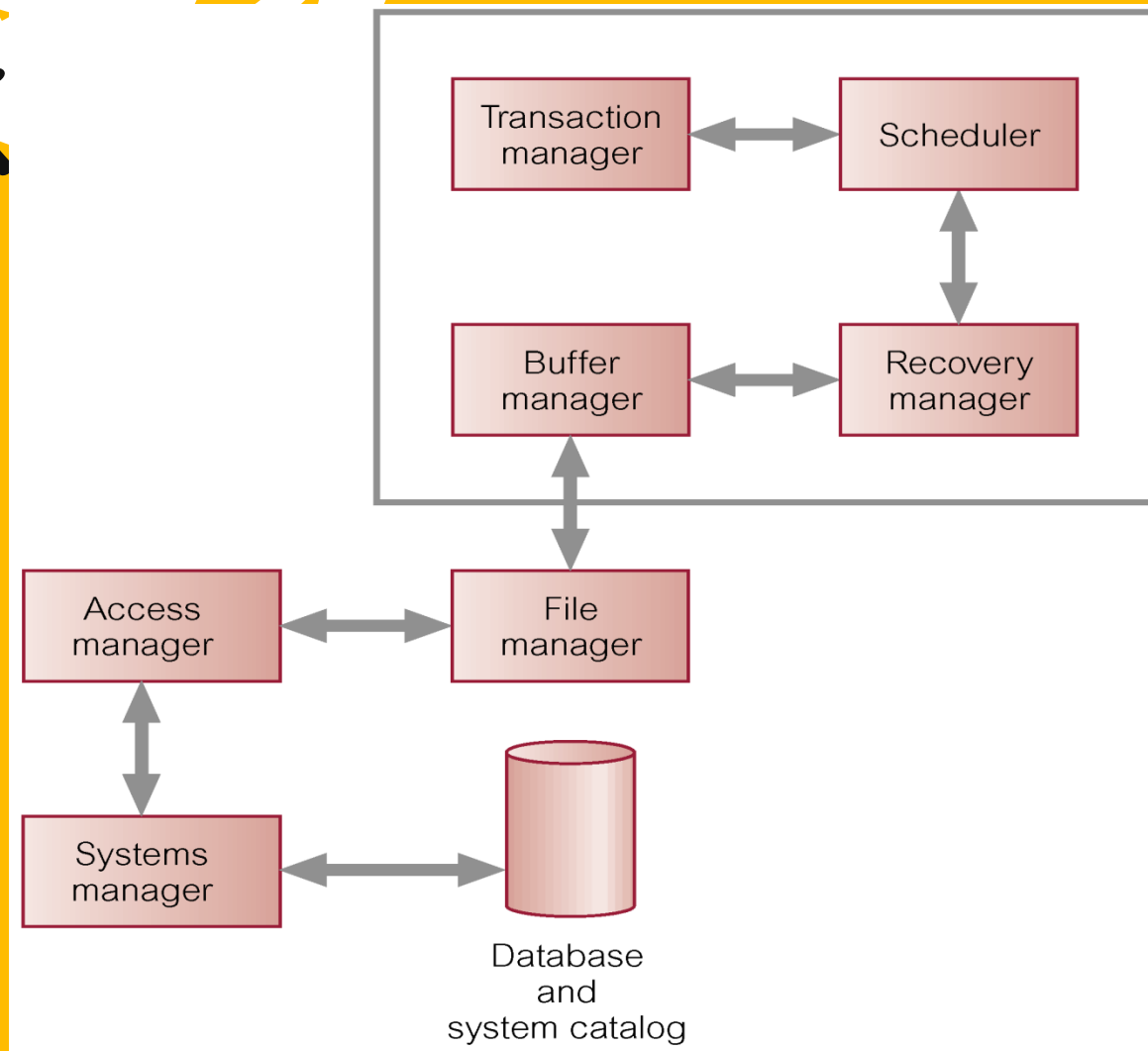
State Transition Diagram for Transaction



Properties of Transaction

Atomicity	'All or nothing' property.
Consistency	Must transform database from one consistent state to another.
Isolation	Partial effects of incomplete transactions should not be visible to other transactions.
Durability	Effects of a committed transaction are permanent and must not be lost because of later failure.

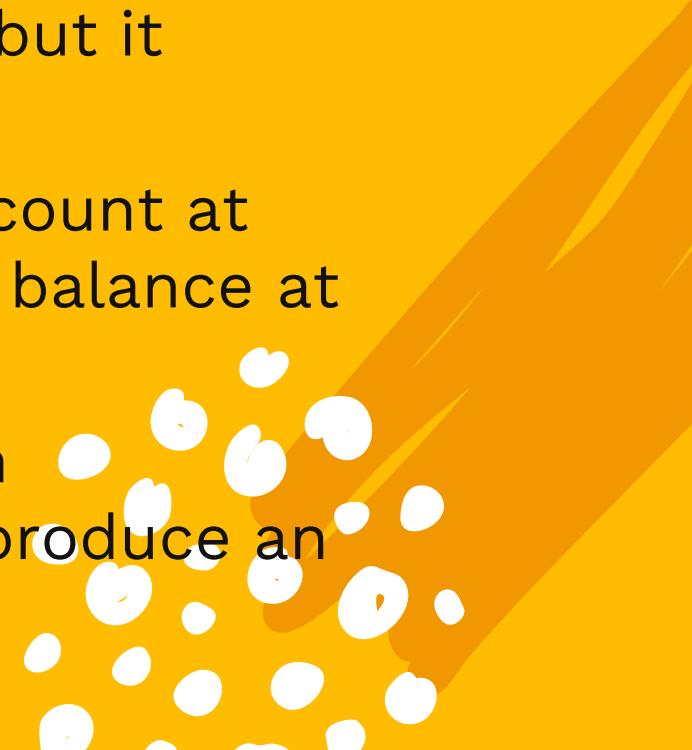
DBMS Transaction Subsystem



- X Transaction manager: Coordinates transaction on behalf of application program. Communicates with scheduler
- X Scheduler: Module responsible for implementing a particular strategy for concurrency control. Sometimes referred to as the lock manager if the concurrency protocol is locking-based.
- X Recovery manager: Ensure database is restored to the state it was in before the start of the transaction and therefore a consistent state.
- X Buffer manager: Responsible for the efficient transfer of data between disk storage and main memory



Concurrency Control

- ✧ Defined as process of managing simultaneous operations on the database without having them interfere with one another.
 - x Transaction can interleave with each other, but it cannot interfere.
 - x If 2 users want to update the same bank account at the same time, there will be some incorrect balance at the end of the transactions.
 - x Although two transactions may be correct in themselves, interleaving of operations may produce an incorrect result.
- 



Need for Concurrency Problem

- x Concurrency control is needed because of, everybody want everything to settle fast with a correct result.
 - x If there is no concurrency control, the problems that might occurs are:
 - x Lost Update Problem
 - x Uncommitted Dependency Problem
 - x Inconsistent Analysis Problem
- 

Lost Update Problem

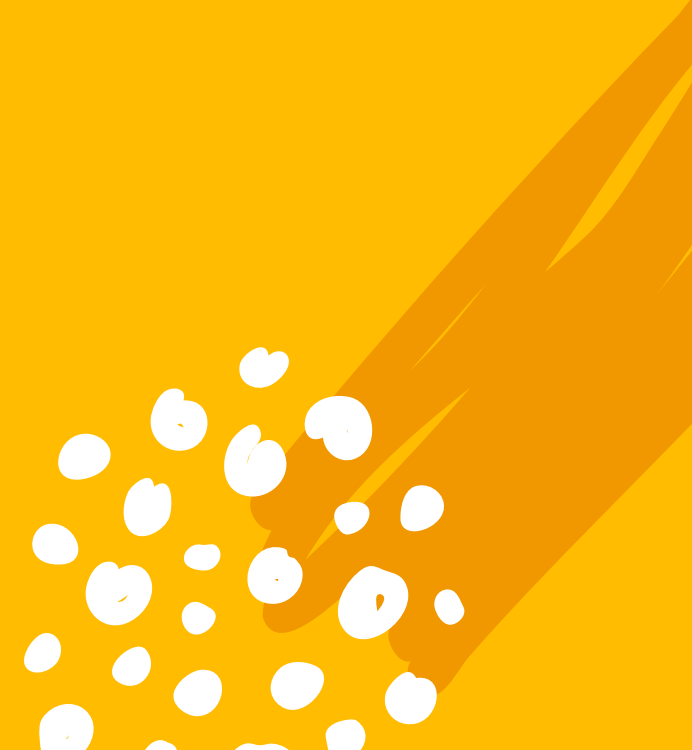
- x Successfully completed update is overridden by another user.

Time	T_1	T_2	bal_x
t_1		begin_transaction	100
t_2	begin_transaction	read(bal_x)	100
t_3	read(bal_x)	$bal_x = bal_x + 100$	100
t_4	$bal_x = bal_x - 10$	write(bal_x)	200
t_5	write(bal_x)	commit	90
t_6	commit		90



Lost Update Problem



- x T1 withdrawing £10 from an account with balx, initially £100.
 - x T2 depositing £100 into same account.
 - x Serially, final balance would be £190.
- 

Uncommitted Dependency Problem

- x Occurs when one transaction can see intermediate results of another transaction before it has committed.

Time	T ₃	T ₄	bal _x
t ₁		begin_transaction	100
t ₂		read(bal_x)	100
t ₃		bal_x = bal_x + 100	100
t ₄	begin_transaction	write(bal_x)	200
t ₅	read(bal_x)	:	200
t ₆	bal_x = bal_x - 10	rollback	100
t ₇	write(bal_x)		190
t ₈	commit		190



Uncommitted Dependency Problem

- x T4 updates balx to £200 but it aborts, so balx should be back at original value of £100.
 - x T3 has read new value of balx (£200) and uses value as basis of £10 reduction, giving a new balance of £190, instead of £90.
- 

Inconsistent Analysis Problem

Occurs when transaction reads several values but second transaction updates some of them during execution of first.

Time	T ₅	T ₆	bal _x	bal _y	bal _z	sum
t ₁		begin_transaction	100	50	25	
t ₂	begin_transaction	sum = 0	100	50	25	0
t ₃	read(bal _x)	read(bal _x)	100	50	25	0
t ₄	bal _x = bal _x - 10	sum = sum + bal _x	100	50	25	100
t ₅	write(bal _x)	read(bal _y)	90	50	25	100
t ₆	read(bal _z)	sum = sum + bal _y	90	50	25	150
t ₇	bal _z = bal _z + 10		90	50	25	150
t ₈	write(bal _z)		90	50	35	150
t ₉	commit	read(bal _z)	90	50	35	150
t ₁₀		sum = sum + bal _z	90	50	35	185
t ₁₁		commit	90	50	35	185

Inconsistent Analysis Problem

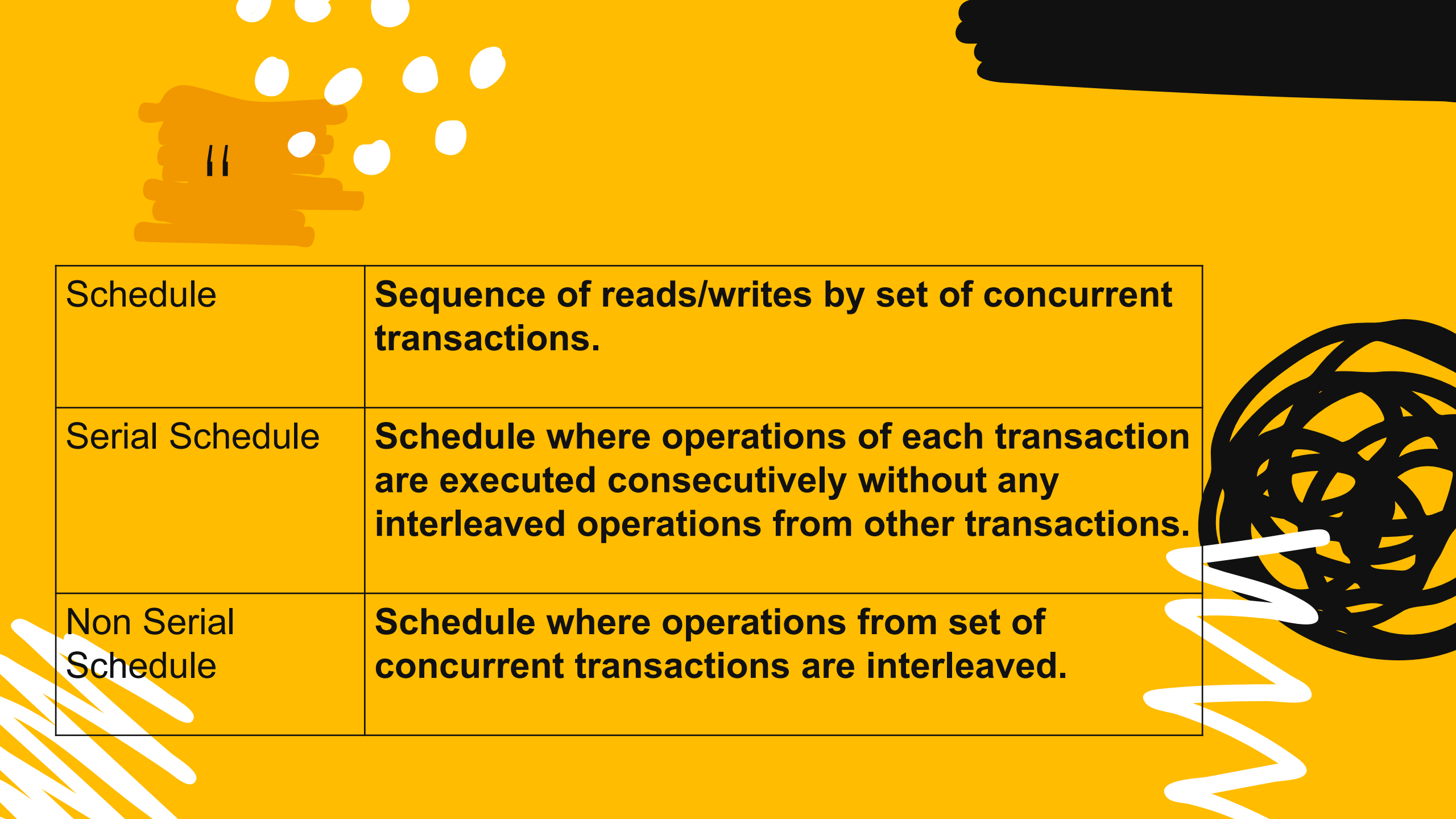
- Sometimes referred to as dirty read or unrepeatable read.
- x T6 is totaling balances of account x (£100), account y (£50), and account z (£25).
- x Meantime, T5 has transferred £10 from balx to balz, so T6 now has wrong result (£10 too high).



Serializability



- x Objective of a concurrency control protocol is to schedule transactions in such a way as to avoid any interference.
 - x Could run transactions serially, but this limits degree of concurrency or parallelism in system.
 - x Serializability identifies those executions of transactions guaranteed to ensure consistency.
- 



Schedule	Sequence of reads/writes by set of concurrent transactions.
Serial Schedule	Schedule where operations of each transaction are executed consecutively without any interleaved operations from other transactions.
Non Serial Schedule	Schedule where operations from set of concurrent transactions are interleaved.

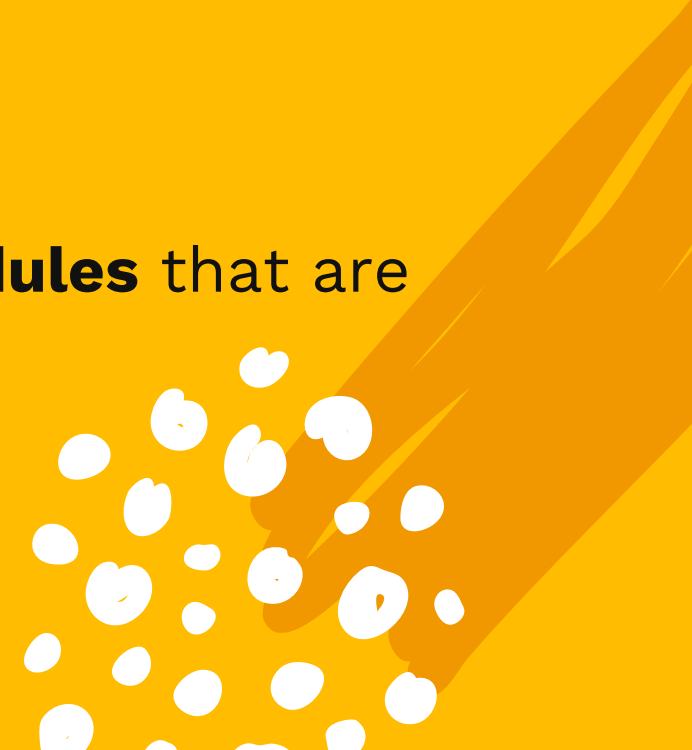


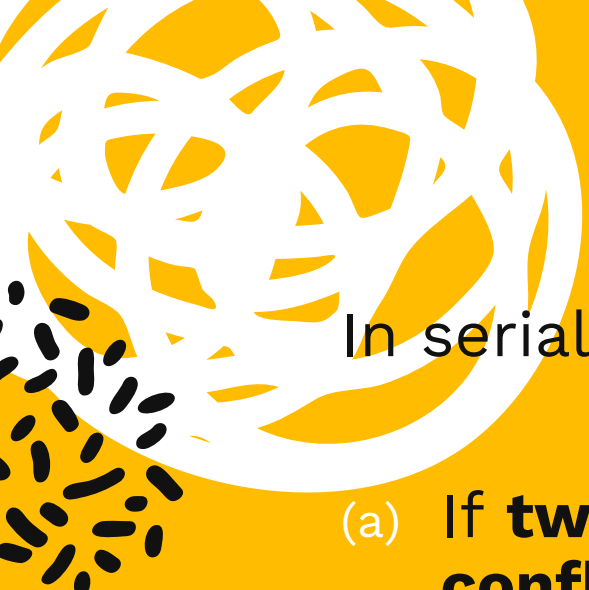
x No guarantee that results of all serial executions of a given set of transactions will be identical.

x Objective of serializability is to find **nonserial schedules** that allow transactions to execute **concurrently without interfering** with one another.

x In other words, want to find **nonserial schedules** that are **equivalent** to *some* **serial schedule**.

x Such a schedule is called *serializable*.





In serializability, **ordering** of read/writes is important:

- (a) If **two transactions only read** a data item, they **do not conflict** and **order is not important**.
 - (b) If **two transactions either read or write** completely **separate data items**, they **do not conflict** and **order is not important**.
 - (c) If **one transaction writes** a data item and **another reads or writes same data** item, **order of execution is important**.
- 

Time	T ₇	T ₈
t ₁	begin_transaction	
t ₂	read(bal_x)	
t ₃	write(bal_x)	
t ₄		begin_transaction
t ₅		read(bal_x)
t ₆		write(bal_x)
t ₇	read(bal_y)	
t ₈	write(bal_y)	
t ₉	commit	
t ₁₀		read(bal_y)
t ₁₁		write(bal_y)
t ₁₂		commit

(a)

	T ₇	T ₈
	begin_transaction	
	read(bal_x)	
	write(bal_x)	
		begin_transaction
		read(bal_x)
	read(bal_y)	
		write(bal_x)
	write(bal_y)	
	commit	
		read(bal_y)
		write(bal_y)
		commit

(b)

	T ₇	T ₈
	begin_transaction	
	read(bal_x)	
	write(bal_x)	
		read(bal_y)
		write(bal_y)
	commit	
		begin_transaction
		read(bal_x)
		write(bal_x)
		read(bal_y)
		write(bal_y)
		commit

(c)

- X Equivalent schedule:
- X (a) Non serial Schedule S1;
- X (b) Non serial schedule S2;
- X (c) Serial schedule S3, equivalent to S1 and S2

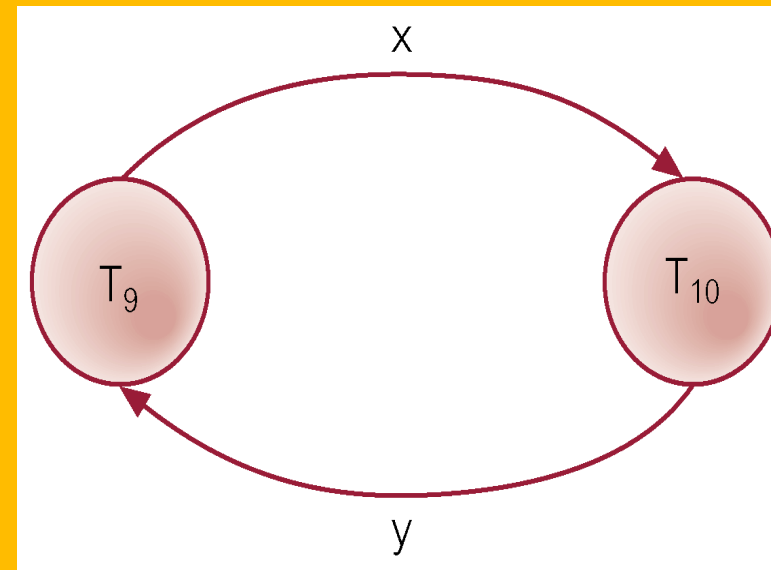
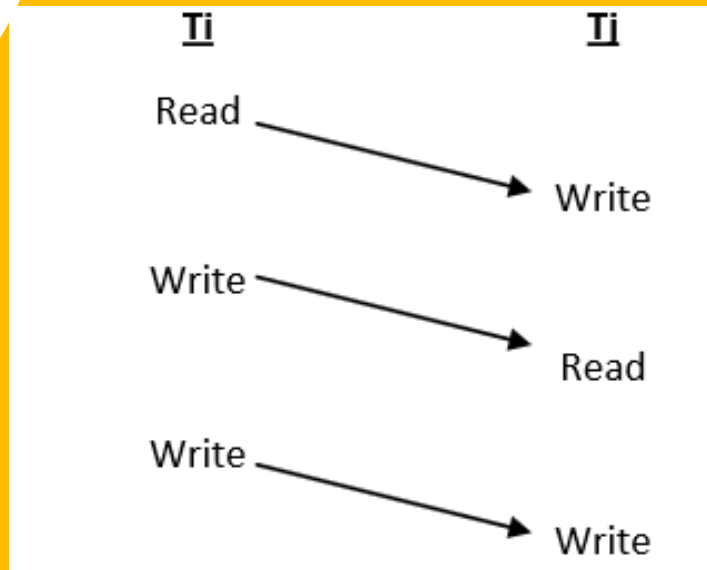
- X Conflict serializable schedule orders any conflicting operations in same way as some serial execution.

Testing for Conflict Serializability

Under constrained write rule (transaction updates data item based on its old value, which is first read), use precedence graph to test for serializability.

X Create:

- X **node** for each transaction;
- X a directed edge $T_i \rightarrow T_j$, if T_j reads the value of an item written by T_i ;
- X a directed edge $T_i \rightarrow T_j$, if T_j writes a value into an item after it has been read by T_i .
- X a directed edge $T_i \rightarrow T_j$, if T_j writes a value into an item after it has been written by T_i .



- x If precedence graph contains cycle schedule is not conflict serializable.

Example on Precedence Graph

$S = [R_1(Z), R_2(Y), W_2(Y), R_3(Y), R_1(X), W_1(X), W_1(Z), W_3(Y), R_2(X), R_1(Y), W_1(Y), W_2(X), R_3(W), W_3(W)]$

- x By using precedence graph, determine whether this transaction is serializable or not.

View Serializability

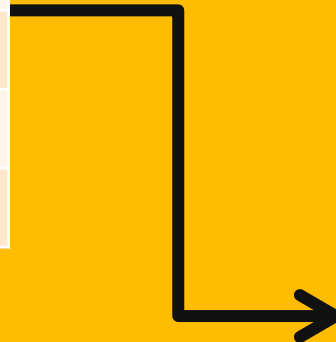
- Offers less stringent definition of schedule equivalence than conflict serializability.
- ✗ Two schedules S_1 and S_2 are view equivalent if:
 - ✗ For each data item x , if T_i reads initial value of x in S_1 , T_i must also read initial value of x in S_2 .
 - ✗ For each read on x by T_i in S_1 , if value read by x is written by T_j , T_i must also read value of x produced by T_j in S_2 .
 - ✗ For each data item x , if last write on x performed by T_i in S_1 , same transaction must perform final write on x in S_2 .

Motivating Example

Determine whether S1 is View Serializable with S2

T1	T2	T3
		Read A
	Read A	
		Write A
Read A		
Write A		

S1



S2		
T2	T3	T1
Read A		
	Read A	
	Write A	
		Read A
		Write A



Rules 1: Initial Read

x Data item A:

S1	S2
T3, T2	T2, T3

x Rules 2: Write-Read Conflict

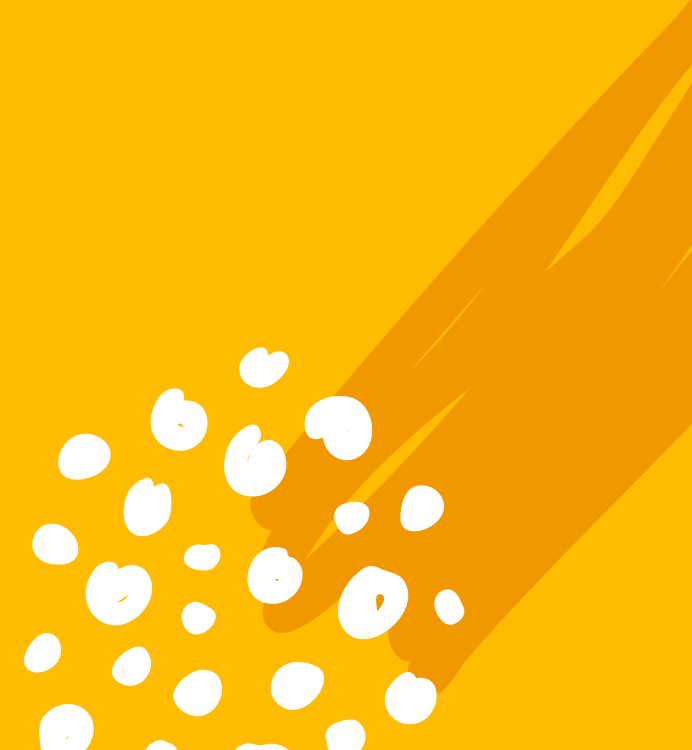
x Data item A:

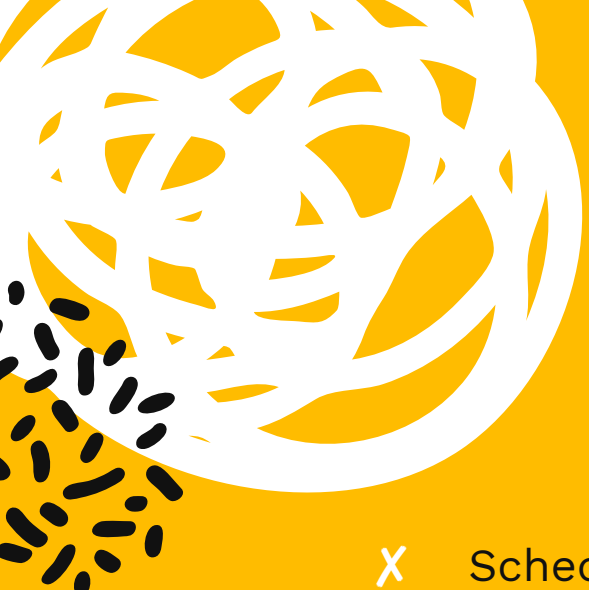
S1	S2
T3 → T1	T3 → T1

x Rules 3: Last Write

x Data item A:

S1	S2
T1	T1



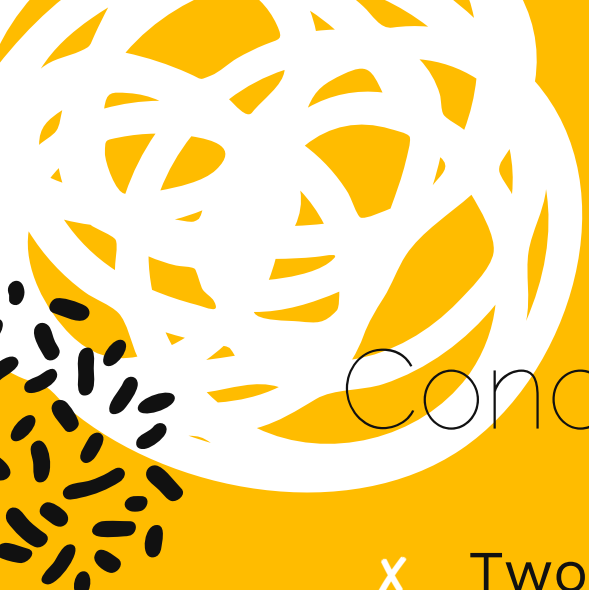
- 
- 
- 
- X Schedule is view serializable if it is view equivalent to a serial schedule.
 - X Every conflict serializable schedule is view serializable, although converse is not true.
 - X It can be shown that any view serializable schedule that is not conflict serializable contains one or more blind writes.
 - X In general, testing whether schedule is serializable is NP-complete (solutions can be verified in polynomial time – computer time).



Recoverability



- X Serializability identifies schedules that maintain database consistency, assuming no transaction fails.
 - X Could also examine recoverability of transactions within schedule.
 - X If transaction fails, atomicity requires effects of transaction to be undone.
 - X Durability states that once transaction commits, its changes cannot be undone (without running another, compensating, transaction).
- 



Concurrency Control Techniques

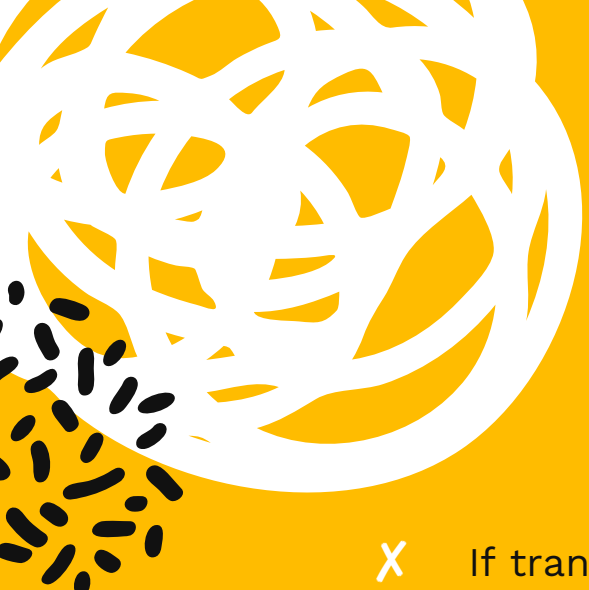
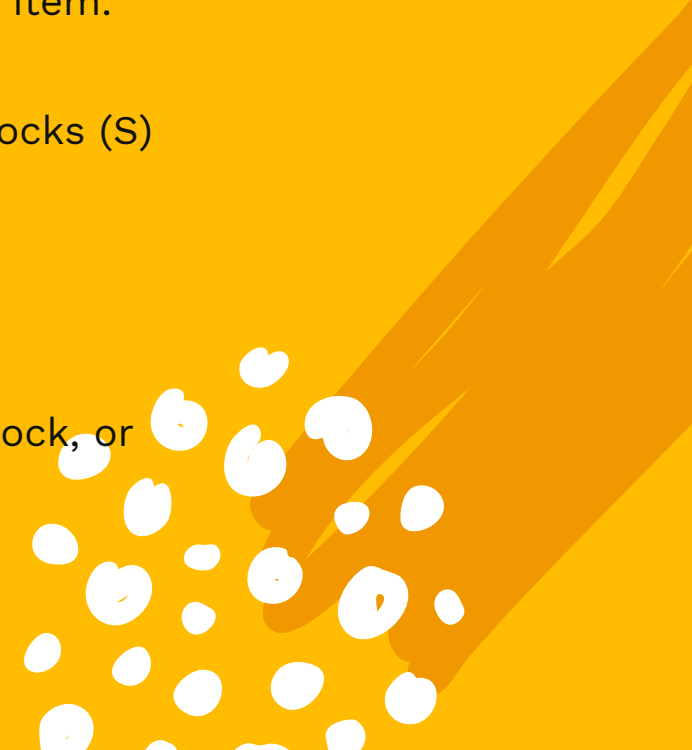
- x Two basic concurrency control techniques:
 - x Locking,
 - x Timestamping.
 - x Both are **conservative** approaches: delay transactions in case they conflict with other transactions.
 - x Optimistic methods **assume conflict is rare** and only check for conflicts at commit.
- 



Locking

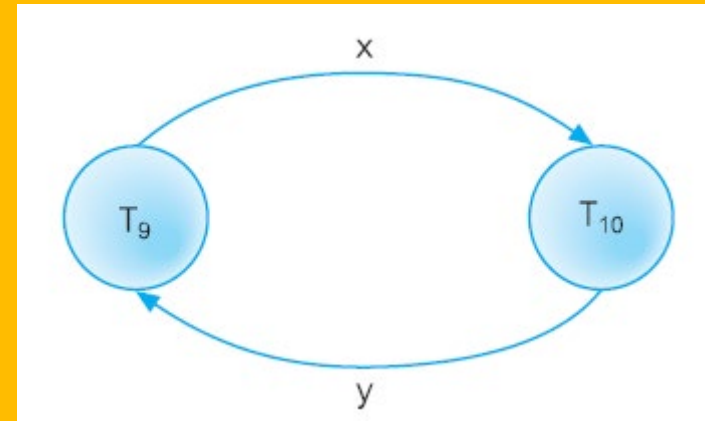


- X Transaction uses locks to deny access to other transactions and to prevent incorrect updates.
 - X Most widely used approach to ensure serializability.
 - X Generally, a transaction must claim a *shared (read)* or *exclusive (write)* lock on a data item before read or write.
 - X Lock prevents another transaction from modifying item or even reading it, in the case of a write lock.
- 

- 
- 
- 
- X If transaction has shared lock (S) on item, can read but not update item.
 - X If transaction has exclusive lock (X) on item, can both read and update item.
 - X **Reads cannot conflict**, so more than one transaction can hold shared locks (S) simultaneously on same item.
 - X Exclusive lock (x) gives transaction **exclusive access** to that item.
 - X Some systems **allow transaction** to upgrade read lock to an exclusive lock, or downgrade exclusive lock to a shared lock.

Locking Method– Incorrect Locking Schedule

Time	T ₉	T ₁₀
t ₁	begin_transaction	
t ₂	read(bal_x)	
t ₃	bal_x = bal_x + 100	
t ₄	write(bal_x)	
t ₅		begin_transaction
t ₆		read(bal_x)
t ₇		bal_x = bal_x * 1.1
t ₈		write(bal_x)
t ₉		read(bal_y)
t ₁₀		bal_y = bal_y * 1.1
t ₁₁	read(bal_y)	write(bal_y)
t ₁₂	bal_y = bal_y - 100	commit
t ₁₃	write(bal_y)	
t ₁₄	commit	



Locking Methods– Incorrect Locking Schedule

X Example - Incorrect Locking Schedule:

For two transactions above, a valid schedule using these rules is:

**S = {write_lock(T₉, bal_x), read(T₉, bal_x), write(T₉, bal_x),
unlock(T₉, bal_x), write_lock(T₁₀, bal_x), read(T₁₀, bal_x), write(T₁₀,
bal_x), unlock(T₁₀, bal_x), write_lock(T₁₀, bal_y), read(T₁₀, bal_y),
write(T₁₀, bal_y), unlock(T₁₀, bal_y), commit(T₁₀), write_lock(T₉,
bal_y), read(T₉, bal_y), write(T₉, bal_y), unlock(T₉, bal_y), commit(T₉)
}**

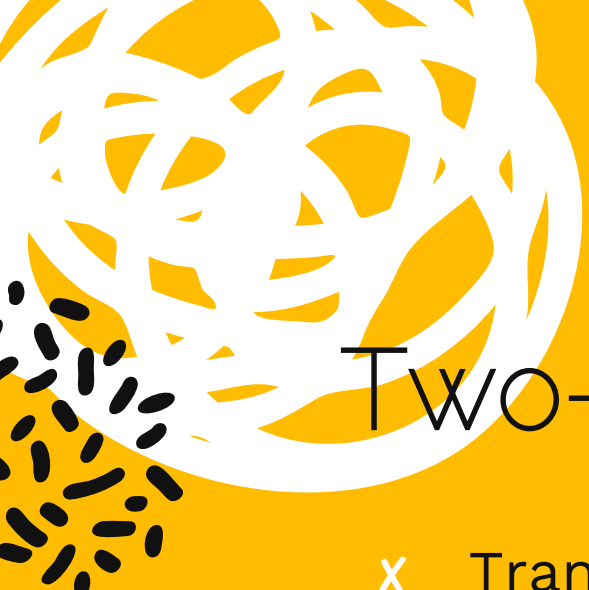
Locking Methods– Incorrect Locking Schedule

- x If at start, $bal_x = 100$, $bal_y = 400$, result should be:
 - x **$bal_x = 220$, $bal_y = 330$, if T_9 executes before T_{10} , or**
 - x **$bal_x = 210$, $bal_y = 340$, if T_{10} executes before T_9 .**
- x **However, result gives $bal_x = 220$ and $bal_y = 340$.**
- x S is **not a serializable** schedule.



Locking Methods – Incorrect Locking Schedule

- x Problem is that **transactions release locks too soon**, resulting in **loss of total isolation and atomicity**.
 - x To guarantee serializability, **need an additional protocol** concerning the positioning of lock and unlock operations in every transaction.
- 



Two-Phase Locking (2PL)

- x Transaction follows 2PL protocol if all locking operations precede first unlock operation in the transaction.
 - x Two phases for transaction:
 - x **Growing phase** - acquires all locks but **cannot release any locks**.
 - x **Shrinking phase** - releases locks but **cannot acquire any new locks**.
- 

Preventing Lost Update Problem using 2PL

Time	T ₁	T ₂	bal _x
t ₁		begin_transaction	100
t ₂	begin_transaction	write_lock(bal_x)	100
t ₃	write_lock(bal_x)	read(bal_x)	100
t ₄	WAIT	bal_x = bal_x + 100	100
t ₅	WAIT	write(bal_x)	200
t ₆	WAIT	commit/unlock(bal_x)	200
t ₇	read(bal_x)		200
t ₈	bal_x = bal_x - 10		200
t ₉	write(bal_x)		190
t ₁₀	commit/unlock(bal_x)		190

Preventing Uncommitted Dependency Problem using 2PL

Time	T ₃	T ₄	bal _x
t ₁		begin_transaction	100
t ₂		write_lock(bal_x)	100
t ₃		read(bal_x)	100
t ₄	begin_transaction	bal_x = bal_x + 100	100
t ₅	write_lock(bal_x)	write(bal_x)	200
t ₆	WAIT	rollback/unlock(bal_x)	100
t ₇	read(bal_x)		100
t ₈	bal_x = bal_x - 10		100
t ₉	write(bal_x)		90
t ₁₀	commit/unlock(bal_x)		90

Preventing Inconsistent Analysis Problem using 2PL

Time	T ₅	T ₆	bal _x	bal _y	bal _z	sum
t ₁		begin_transaction	100	50	25	
t ₂	begin_transaction	sum = 0	100	50	25	0
t ₃	write_lock(bal _x)		100	50	25	0
t ₄	read(bal _x)	read_lock(bal _x)	100	50	25	0
t ₅	bal _x = bal _x - 10	WAIT	100	50	25	0
t ₆	write(bal _x)	WAIT	90	50	25	0
t ₇	write_lock(bal _z)	WAIT	90	50	25	0
t ₈	read(bal _z)	WAIT	90	50	25	0
t ₉	bal _z = bal _z + 10	WAIT	90	50	25	0
t ₁₀	write(bal _z)	WAIT	90	50	35	0
t ₁₁	commit/unlock(bal _x , bal _z)	WAIT	90	50	35	0
t ₁₂		read(bal _x)	90	50	35	0
t ₁₃		sum = sum + bal _x	90	50	35	90
t ₁₄		read_lock(bal _y)	90	50	35	90
t ₁₅		read(bal _y)	90	50	35	90
t ₁₆		sum = sum + bal _y	90	50	35	140
t ₁₇		read_lock(bal _z)	90	50	35	140
t ₁₈		read(bal _z)	90	50	35	140
t ₁₉		sum = sum + bal _z	90	50	35	175
t ₂₀		commit/unlock(bal _x , bal _y , bal _z)	90	50	35	175

Cascading Rollback

Time	T ₁₄	T ₁₅	T ₁₆
t ₁	begin_transaction		
t ₂	write_lock(bal_x)		
t ₃	read(bal_x)		
t ₄	read_lock(bal_y)		
t ₅	read(bal_y)		
t ₆	bal_x = bal_y + bal_x		
t ₇	write(bal_x)		
t ₈	unlock(bal_x)	begin_transaction	
t ₉	:	write_lock(bal_x)	
t ₁₀	:	read(bal_x)	
t ₁₁	:	bal_x = bal_x + 100	
t ₁₂	:	write(bal_x)	
t ₁₃	:	unlock(bal_x)	
t ₁₄	:	:	
t ₁₅	rollback	:	
t ₁₆		:	begin_transaction
t ₁₇		:	read_lock(bal_x)
t ₁₈		rollback	:
t ₁₉			rollback

X If every transaction in a schedule follows 2PL, schedule is serializable.

X However, problems can occur with interpretation of when locks can be released.



Cascading Rollback

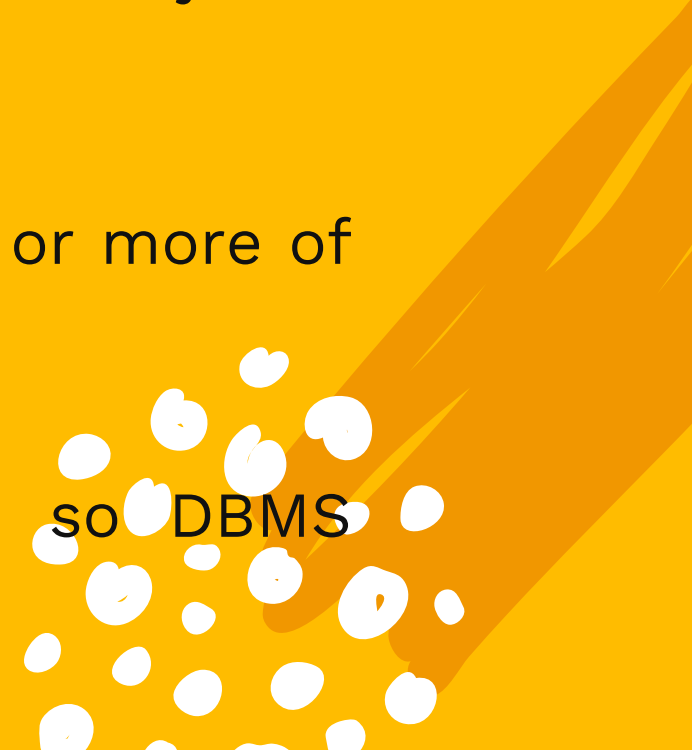


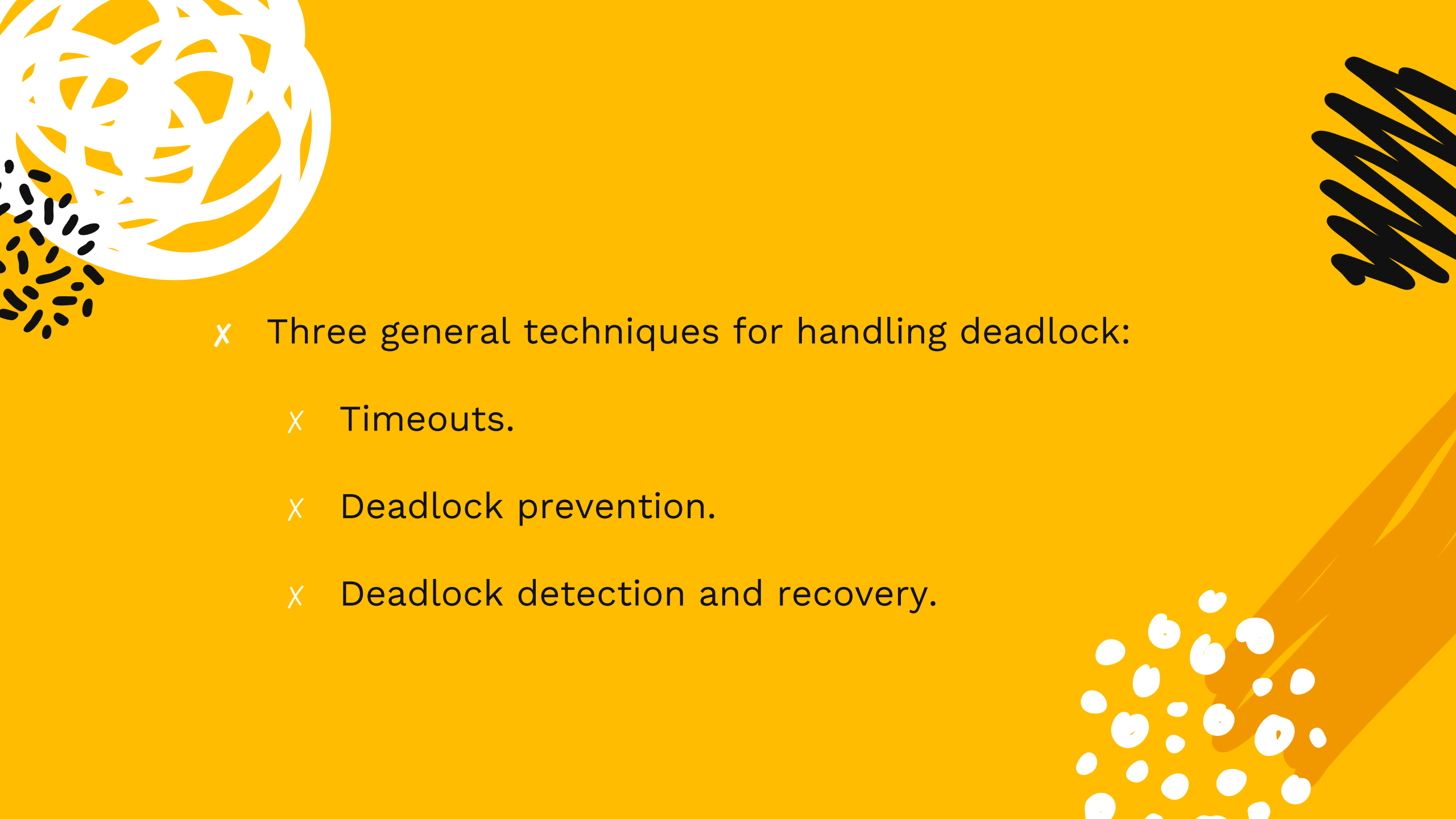
- x Transactions conform to 2PL.
 - x T14 aborts.
 - x Since T15 is dependent on T14, T15 must also be rolled back. Since T16 is dependent on T15, it too must be rolled back.
 - x This is called cascading rollback.
 - x To prevent this with 2PL, leave release of all locks until end of transaction.
- 



Deadlock



- x An impasse that may result when two (or more) transactions are each waiting for locks held by the other to be released.
 - x Only one way to break deadlock: abort one or more of the transactions.
 - x Deadlock should be transparent to user, so DBMS should restart transaction(s).
- 

- 
- x Three general techniques for handling deadlock:
 - x Timeouts.
 - x Deadlock prevention.
 - x Deadlock detection and recovery.



Deadlock - Timeouts

- x Transaction that requests lock will only wait for a system-defined period of time.
 - x If lock has not been granted within this period, lock request times out.
 - x In this case, DBMS assumes transaction may be deadlocked, even though it may not be, and it aborts and automatically restarts the transaction.
- 

Deadlock - Prevention

- x DBMS looks ahead to see if transaction would cause deadlock and never allows deadlock to occur.
- x Could order transactions using transaction timestamps:

Wait-Die

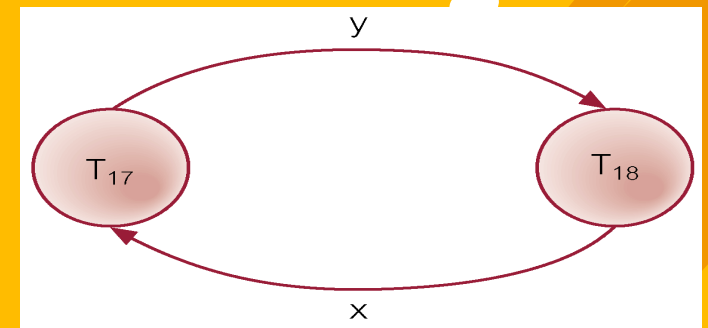
- only an older transaction can wait for younger one, otherwise transaction is aborted (dies) and restarted with same timestamp

Wound-Wait

- only a younger transaction can wait for an older one. If older transaction requests lock held by younger one, younger one is aborted (wounded).

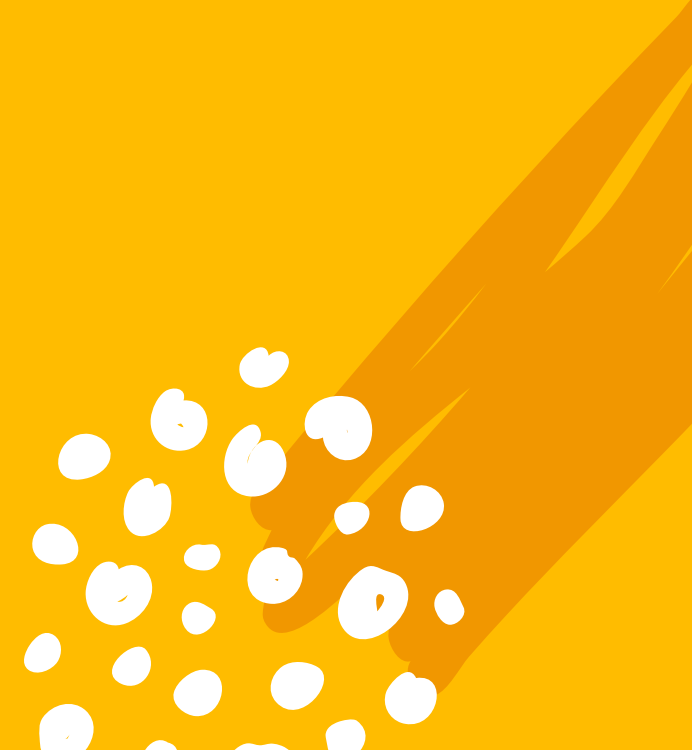
Deadlock Detection and Recovery

- X DBMS allows deadlock to occur but recognizes it and breaks it.
- X Usually handled by construction of wait-for graph (WFG) showing transaction dependencies:
 - X Create a node for each transaction.
 - X Create edge $T_i \rightarrow T_j$, if T_i waiting to lock item locked by T_j .
- X Deadlock exists if and only if WFG contains cycle.
- X WFG is created at regular intervals.





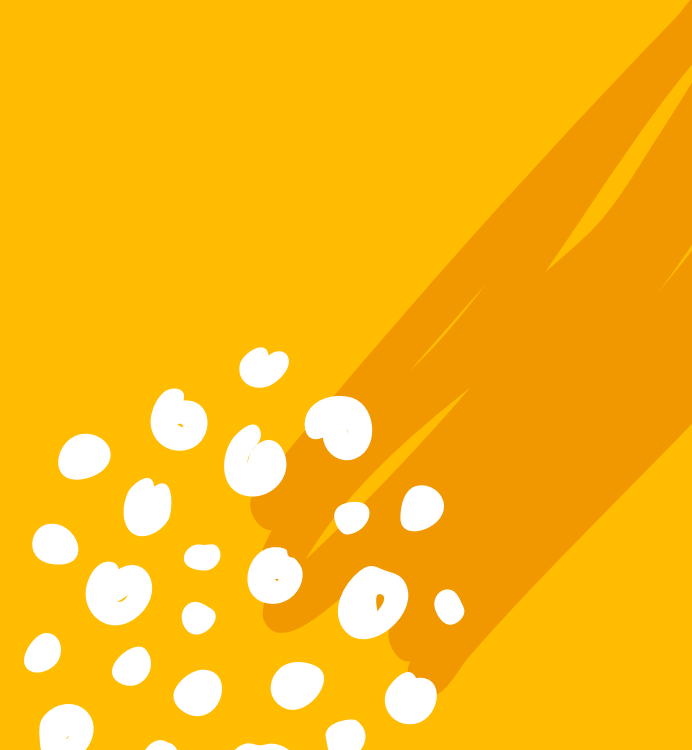
Recovery from Deadlock Detection

- x Several issues:
 - x choice of deadlock victim;
 - x how far to roll a transaction back;
 - x avoiding starvation.
- 



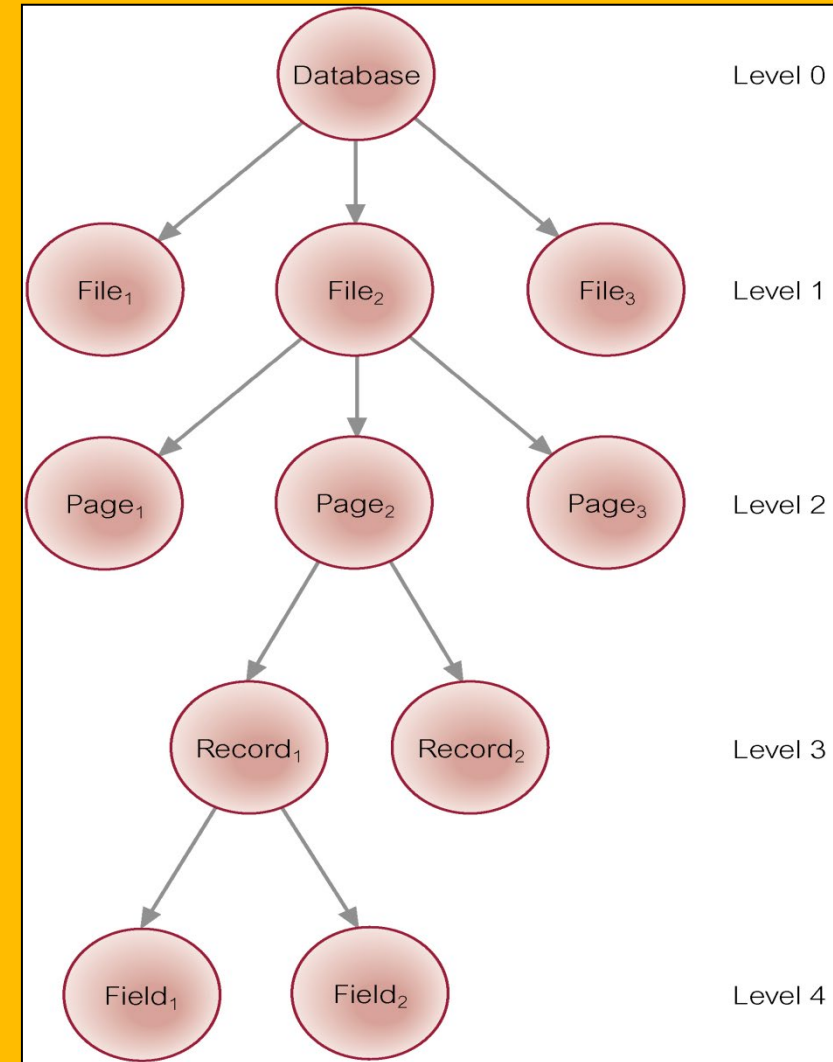
Granularity of Data Items



- x Granularity - size of data items chosen as unit of protection by concurrency control protocol.
 - x Ranging from coarse to fine:
 - x The entire database.
 - x A file.
 - x A page (or area or database spaced).
 - x A record.
 - x A field value of a record.
- 

Granularity of Data Items

- x When node is locked, all its descendants are also locked.

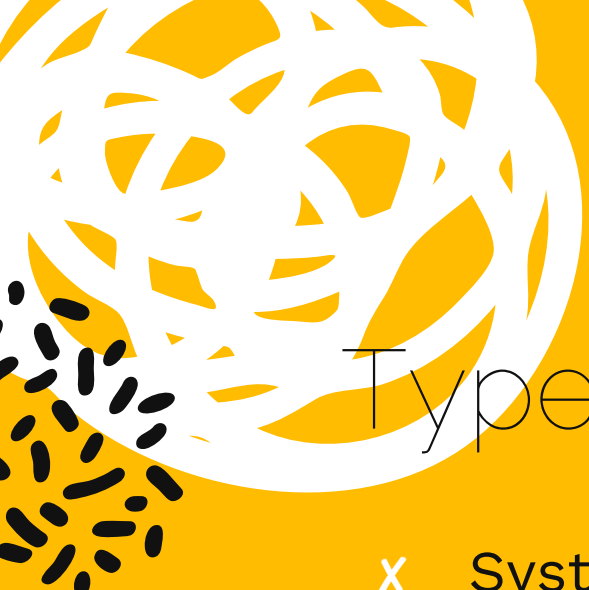




Database Recovery



- X No matter what, assume that your data will corrupt one day and you need a mechanism to reconstruct back the data lost.
 - X Database Recovery: Process of **restoring** database to a **correct state** in the **event of a failure**.
 - X Need for Recovery Control
 - X Two types of storage: volatile (main memory) and nonvolatile.
 - X Volatile storage does not survive system crashes.
 - X Stable storage represents information that has been replicated in several nonvolatile storage media with independent failure modes.
- 



Types of Failure

- x System crashes, resulting in loss of main memory.
 - x Media failures, resulting in loss of parts of secondary storage.
 - x Application software errors.
 - x Natural physical disasters.
 - x Carelessness or unintentional destruction of data or facilities.
 - x Sabotage, or intentional corruption or destruction of data, hardware or software facilities
- 

Recovery Facilities

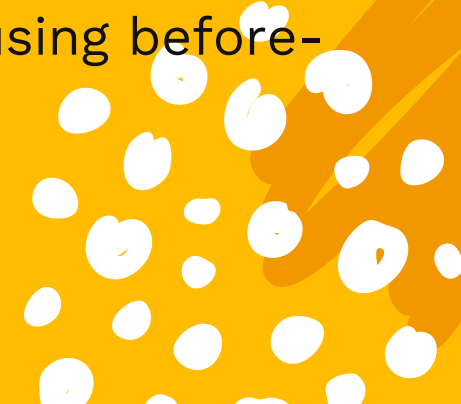
Backing up the database

- X On a regularly scheduled basis, a company's databases must be backed up or copied.
 - X The copy version must be put away from original copy
-
- X Maintaining a journal (Log File)- Keep track the changes to the data such as INSERTION, UPDATE and DELETION
 - X Checkpoint facility, which enables updates to database in progress to be made permanent.
 - X Recovery manager, which allows DBMS to restore database to consistent state following a failure.



Recovery Techniques

- X If database has been **damaged**:
 - X Need to restore last backup copy of database and reapply updates of committed transactions using log file.

 - X If database is only **inconsistent**:
 - X Need to undo changes that caused inconsistency. May also need to redo some transactions to ensure updates reach secondary storage.
 - X Do not need backup, but can restore database using before- and after-images in the log file.
- 

Main Recovery Techniques

Deferred Update

- Updates are not written to the database until after a transaction has reached its commit point.
- If transaction fails before commit, it will not have modified database and so no undoing of changes required.
- May be necessary to redo updates of committed transactions as their effect may not have reached database.

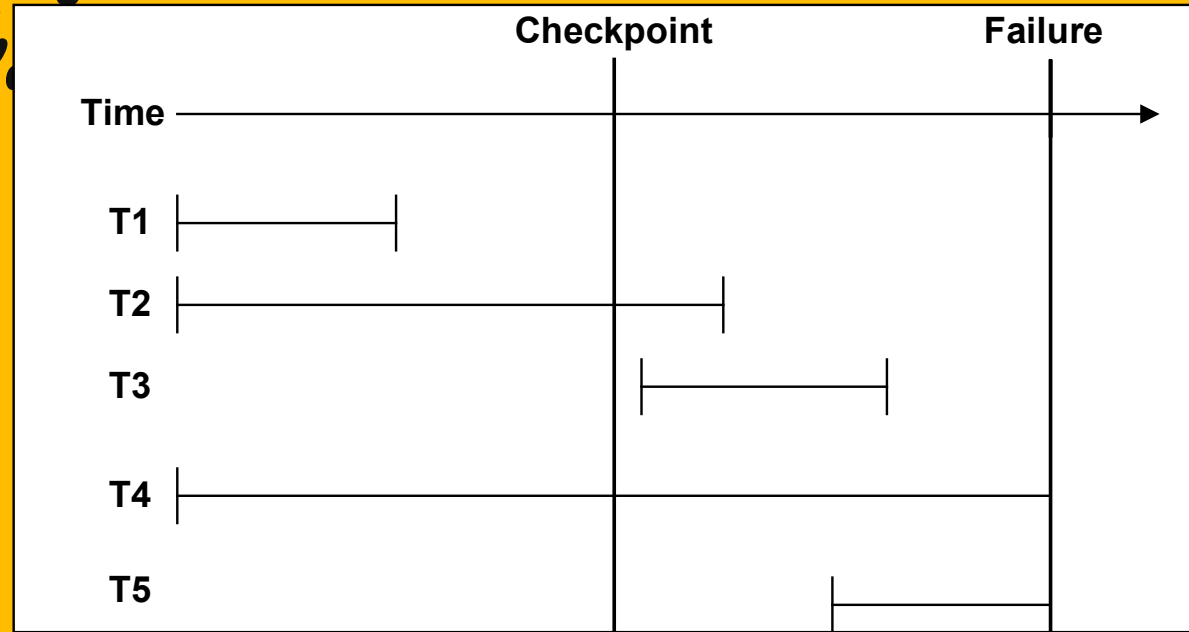
Immediate Update

- Updates are applied to database as they occur.
- Need to redo updates of committed transactions following a failure.
- May need to undo effects of transactions that had not committed at time of failure.

Shadow Paging

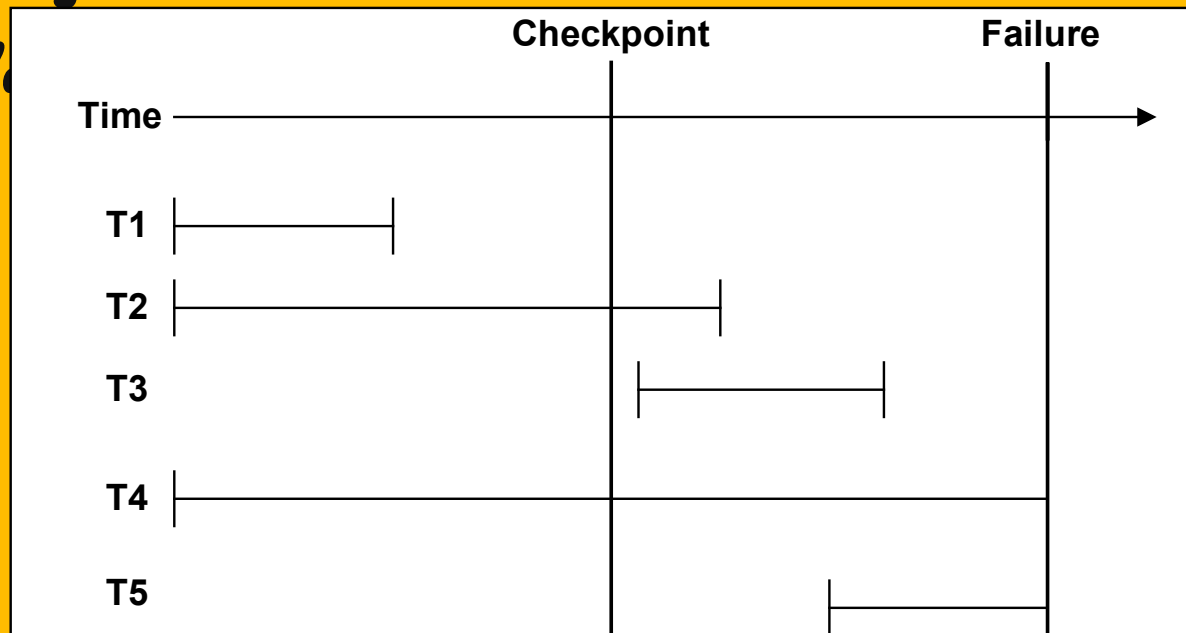
- Maintain two page tables during life of a transaction: *current* page and *shadow* page table.
- When transaction starts, two pages are the same.
- Shadow page table is never changed thereafter and is used to restore database in event of failure.
- During transaction, current page table records all updates to database.
- When transaction completes, current page table becomes shadow page table.

Deferred Update



Class	Description	Restart Work
T1	Finished before CP	None
T2	Started before CP; finished before failure	Redo forward from first log record
T3	Started after CP; finished before failure	Redo forward from first log record
T4	Started before CP; not yet finished	None
T5	Started after CP; not yet finished	None

Immediate Update



Class	Description	Restart Work
T1	Finished before CP	None
T2	Started before CP; finished before failure	Redo forward from checkpoint
T3	Started after CP; finished before failure	Redo forward from checkpoint
T4	Started before CP; not yet finished	Undo backwards from most recent log record
T5	Started after CP; not yet finished	Undo backwards from most recent log record



References

- x Thomas Connolly and Carolyn Begg, Database Systems: A Practical Approach to Design, Implementation, and Management, 6th Edition, Pearson, 2015, ISBN: 978-01329432
- 